Open Source Vulnerability Risk Analysis Pipeline

Dhruv Bhardwaj

April 5, 2026

1 Research Question

This project investigates whether package-level metadata, such as popularity and maintenance activity, is associated with vulnerability exposure in open-source software. In particular, we examine whether widely used or actively maintained packages are more likely to exhibit high-severity vulnerabilities.

To address this question, vulnerability data from the OSV (Open Source Vulnerabilities) database is integrated with package metadata from Libraries.io, enabling a combined analysis of software risk and ecosystem characteristics

2 Methodology

The analysis was conducted using an end-to-end data pipeline consisting of the following stages:

- **Data Acquisition:** Package metadata was collected from Libraries.io, while vulnerability data was retrieved from the OSV API.
- **Data Cleaning and Transformation:** OSV JSON records were flattened and normalized. Missing severity values were approximated using categorical mappings (e.g., HIGH $\rightarrow$ 8.0).
- **Aggregation:** Vulnerability records were aggregated into package-level metrics, including total vulnerability count, high-severity count, and maximum severity score.
- **Data Integration:** The aggregated vulnerability dataset was joined with package metadata.
- **Storage:** The final dataset was stored in a PostgreSQL database for structured querying and analysis.
- **Visualization:** Analytical charts were generated using Python (pandas and matplotlib).

This pipeline ensures reproducibility from raw data ingestion to final analysis.

2.1 Datasets

The datasets used in this project are publicly available:

- OSV (Open Source Vulnerabilities): `https://osv.dev`
- Libraries.io: `https://libraries.io`

2.2 Data Management

A data dictionary was created to document all fields and transformations used in the pipeline. Additionally, a strict separation between raw and cleaned data was maintained:

- Raw data (JSON/CSV) was stored in a dedicated raw data layer
- Cleaned and aggregated datasets were stored separately for analysis

This ensures reproducibility and traceability across all transformation stages.

2.3 Applied Task: Risk Scoring Model

To extend beyond basic aggregation, we construct a composite risk score to quantify the potential impact of vulnerabilities at the package level.

The score combines three factors:

- Vulnerability Count (frequency of issues)
- Maximum Severity Score (worst-case impact)
- Popularity (GitHub stars as a proxy for exposure)

The intuition is that widely used packages with severe vulnerabilities pose a disproportionately higher risk to the ecosystem.

A simplified scoring formulation is used:

$$RiskScore = VulnerabilityCount \times MaxSeverity \times \log(Stars + 1)$$

This formulation emphasizes both severity and ecosystem exposure while reducing skew using logarithmic scaling.

3 Results

A total of 2414 packages were analyzed after data cleaning and integration. The results highlight several key relationships between package characteristics and vulnerability exposure. This analysis focuses on extracting interpretable patterns and relationships from the data, forming a foundation for future predictive modeling.

3.1 Vulnerability Distribution

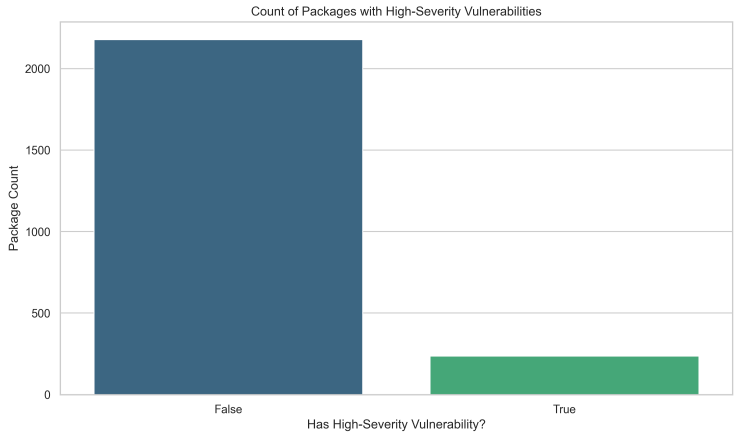

Figure 1: Distribution of packages with and without high-severity vulnerabilities.

Figure 1 shows that 13.5% of packages contain at least one known vulnerability, while 9.8% contain at least one high-severity vulnerability. This indicates that severe vulnerabilities are relatively concentrated among a subset of packages.

3.2 Severity Characteristics

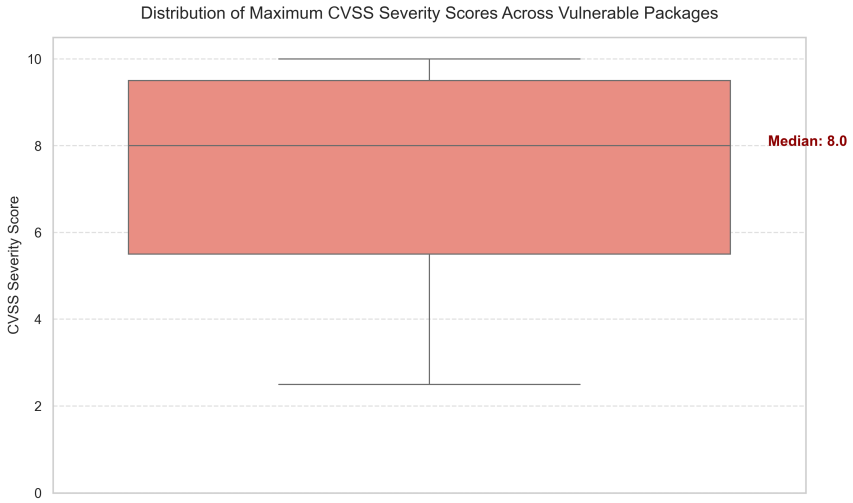

Figure 2: Distribution of maximum CVSS severity scores across vulnerable packages.

Figure 2 illustrates that vulnerability severity scores are skewed toward higher values, with many packages exhibiting high or critical severity levels.

3.3 Popularity and Vulnerability Risk

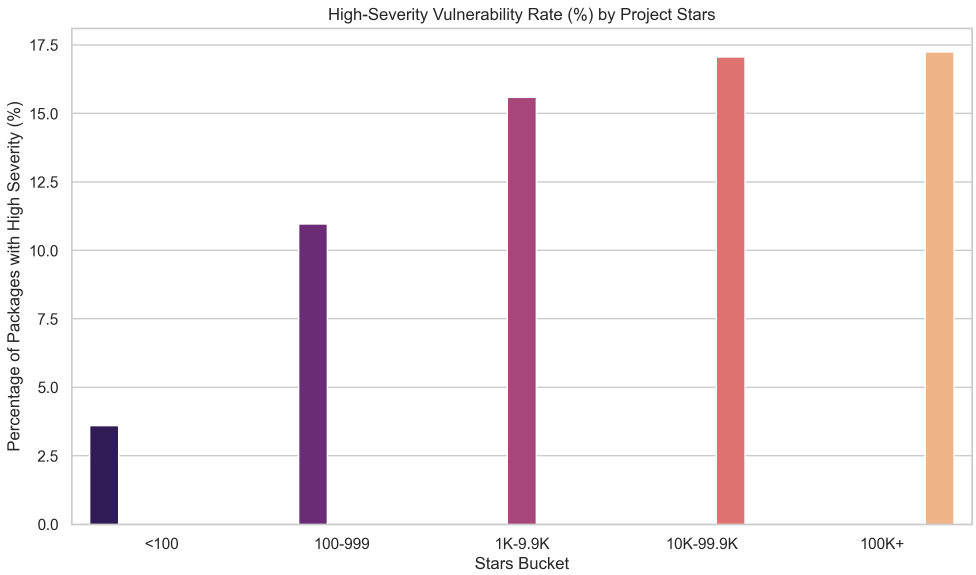

Figure 3: High-severity vulnerability rate by package popularity (stars).

Figure 3 shows a strong upward trend in high-severity vulnerability rates as package popularity increases. Packages with fewer than 100 stars exhibit negligible risk, while those with over 100K stars show rates exceeding 38%. This suggests that widely adopted packages are more likely to accumulate or expose critical vulnerabilities.

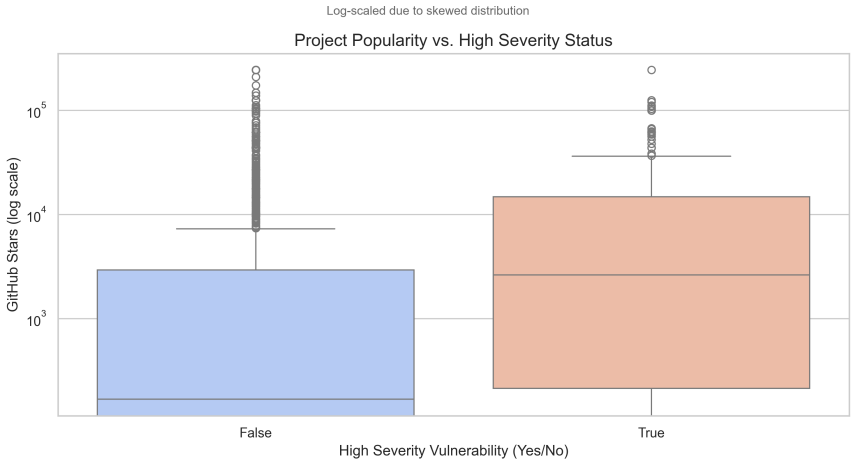

Figure 4: Distribution of GitHub stars by high-severity vulnerability presence (log scale).

Figure 4 further reinforces this relationship, showing that packages with high-severity vulnerabilities have significantly higher star counts on average. A logarithmic scale is used due to the highly skewed distribution of star counts.

3.4 Maintenance Activity

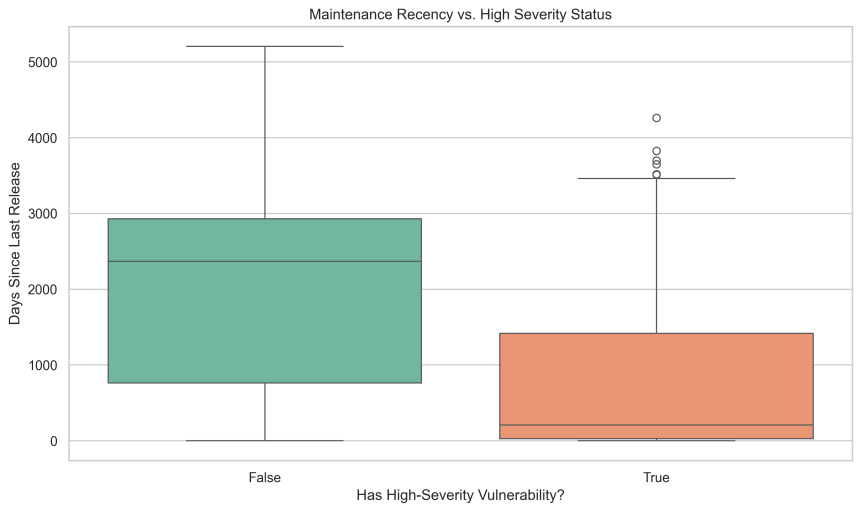

Figure 5: Distribution of days since last release by vulnerability status.

Figure 5 shows that packages with high-severity vulnerabilities are, on average, more recently updated than those without. This suggests that vulnerability exposure is not limited to abandoned projects, but is also prevalent in actively maintained and widely used packages, likely due to increased scrutiny and reporting.

3.5 High-Risk Packages

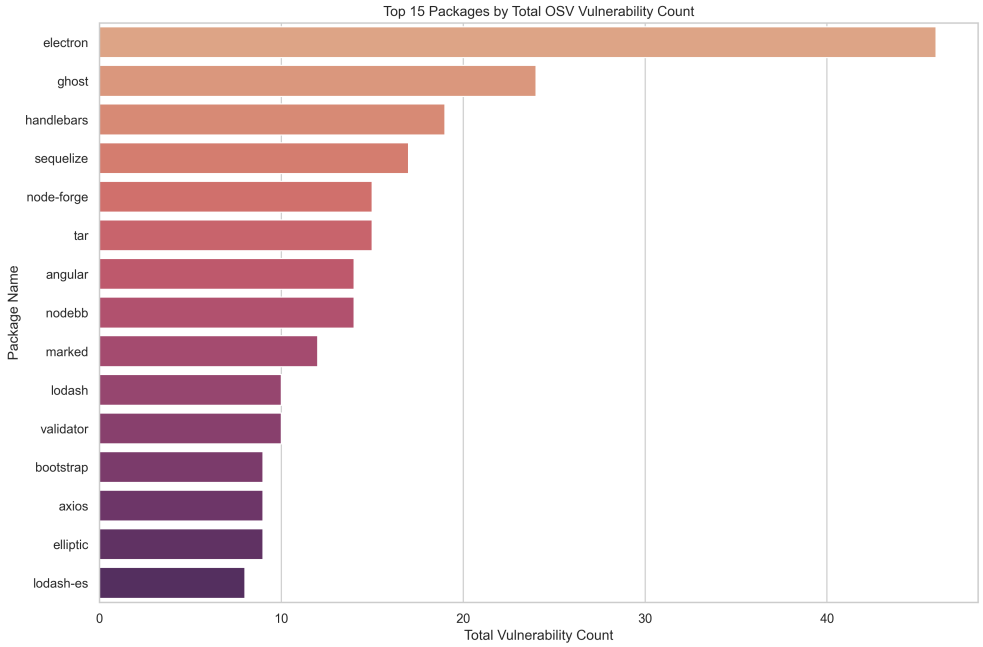

Figure 6: Top packages by total vulnerability count.

Figure 6 highlights packages with the highest number of vulnerabilities. These are typically large, widely used frameworks, suggesting that system complexity and ecosystem importance contribute to increased vulnerability exposure.

3.6 Risk-Based Package Ranking

To move beyond individual metrics, we construct a composite risk score that combines vulnerability frequency, severity, and ecosystem exposure.

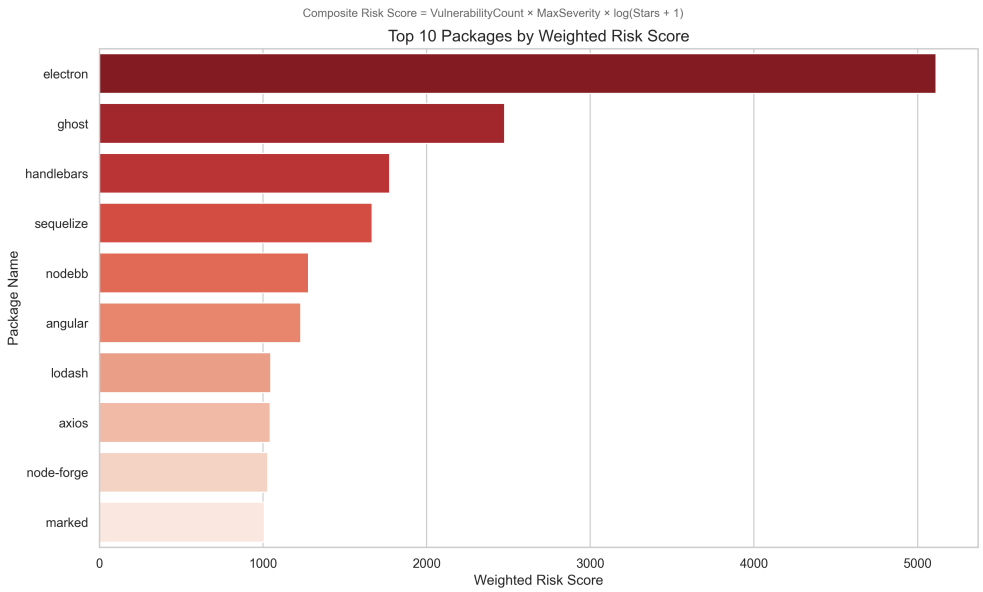

Figure 7: Top packages ranked by composite risk score.

Figure 7 highlights the packages with the highest overall risk. Unlike raw vulnerability counts, this ranking emphasizes projects that combine severe vulnerabilities with widespread adoption.

Highly popular packages such as `electron` and `ghost` rank prominently, indicating that ecosystem risk is driven not only by the presence of vulnerabilities, but also by their potential impact across a large user base.

4 Limitations

Several limitations should be considered when interpreting these results:

- **Reporting Bias:** Popular packages are more likely to be scrutinized, leading to higher recorded vulnerability counts.
- **Data Coverage:** Only publicly disclosed vulnerabilities in the OSV database are included.
- **Severity Approximation:** Some severity values were estimated when official CVSS scores were unavailable.

5 Next Steps

Future work could extend this analysis in several ways:

- Incorporating dependency graph data to analyze transitive vulnerability risk.
- Performing temporal analysis to study how vulnerabilities evolve over time.
- Exploring predictive models to estimate vulnerability risk based on package characteristics.

6 Conclusion

This project demonstrates a complete data analytics pipeline for evaluating open-source software risk. The findings suggest that vulnerability exposure is strongly associated with package popularity and ecosystem visibility, rather than solely with abandonment or lack of maintenance.